SISTEMAS OPERATIVOS

Aula Teórica 14

Daniel Castro
2025/2026

UAlg
UNIVERSIDADE DO ALGARVE

Memory Management (1)

1. Basic memory management
2. Swapping
3. Virtual memory
4. Page replacement algorithms
5. Modeling page replacement algorithms
6. Design issues for paging systems
7. Implementation issues
8. Segmentation

Memory Management (2)

Parkinson's Law: " Programs expand to fill the memory available to hold them"

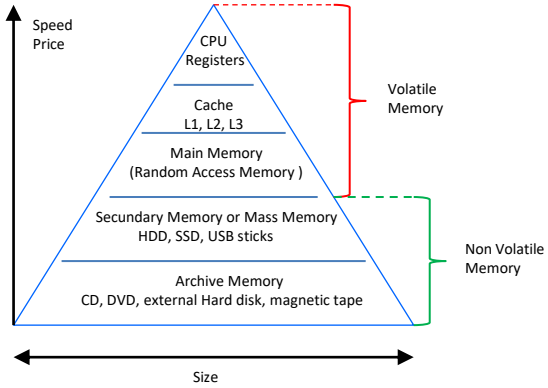
Ideally programmers want memory that is:

- Large
- Fast
- Non volatile

Memory hierarchy

- Registers (1 clock cycle)
- Small amount of fast, expensive memory – cache L1, L2, L3
- Main memory, medium speed, medium price
- Mass memory, slow, high capacity (several Gbyte), cheap

Memory manager handles the memory hierarchy.



CPU Examples:	L1 Data	L1 Instr.	L2	L3	CPU Clock	n. bit	Core/Threads	Year
Pentium IV Extreme	8 Kbytes	8Kbytes	512 Kbytes	2 Mbytes	3.4 GHz	64	1	2004
Intel Sandy Bridge (Core i3, i5, i7, 2nd gen)	32 Kb	32Kb	256 Kb each core	8 Mb shared	3.4 GHz	64	6/12	2011>
AMD Ryzen 7 6800H	256 Kb	256 Kb	4 Mb	16 Mb shared	3.2 > 4.7GHz	64	8/16	2022>

Basic Memory Management – Monoprogramming without Swapping or Paging

The simplest memory abstraction is to have no abstraction at all.
Early mainframe computers (before 1960), early minicomputers (before 1970), early personel computers (before 1980) add no memory abstraction. All programs simply saw the physical memory.

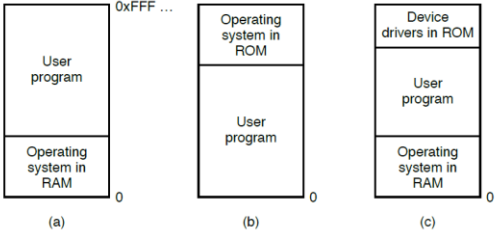


Figure 3-1. Three simple ways of organizing memory with an operating system and one user process. Other possibilities also exist.

Basic Memory Management – Monoprogramming without Swapping or Paging

The simplest memory abstraction is to have no abstraction at all.
Early mainframe computers (before 1960), early minicomputers (before 1970), early personel computers (before 1980) add no memory abstraction. All programs simply saw the physical memory.

Mainframes, Minicomputers

Handheld Computers, Embedded Systems

Early Personal Computers (running MS-DOS), the portion of the system in the ROM is called the BIOS (Basic Input Output System)

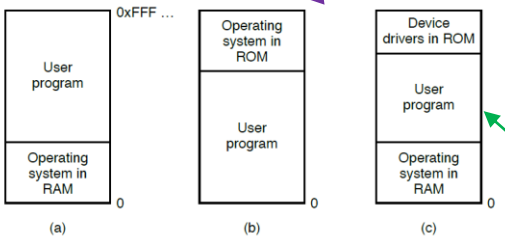


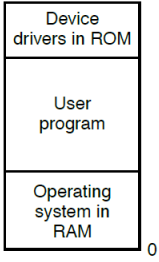
Figure 3-1. Three simple ways of organizing memory with an operating system and one user process. Other possibilities also exist.

Memory Management (4)

In a system like this, generally only one process at a time can be running.
As soon as the user types a command, the operating system copies the requested program from nonvolatile storage to memory and executes it.
When the process finishes, the operating system displays a prompt character (A>) and waits for a new command.

After receiving the users new comand, the operating system will load a new program into the memory, overwriting the first one.

A management model like this have the disadvantage that a bug in the user program can wipe out the operating system, possibly with disastrous results.



Early Personal Computers (running MS-DOS), the portion of the system in the ROM is called the BIOS (Basic Input Output System)

Running Multiple Programs Without a Memory Abstraction

In early model of IBM 360, a technique known **as static relocation**, was implemented, modifying on the fly a second program loaded, adding an address constant to every program addresses during the load.

Example:
We have two 16-Kb programs, in a multiprogram system both are loaded, one after the other.

Without relocation both programs reference absolute physical Memory, the second program will fails on the JMP 28 instruction, because it will be jumping to the ADD instruction in the first Program, insted of the CMP instruction it is supposed to jump to.

With relocation the same instruction will be JMP 16412 (JMP 28+16380).

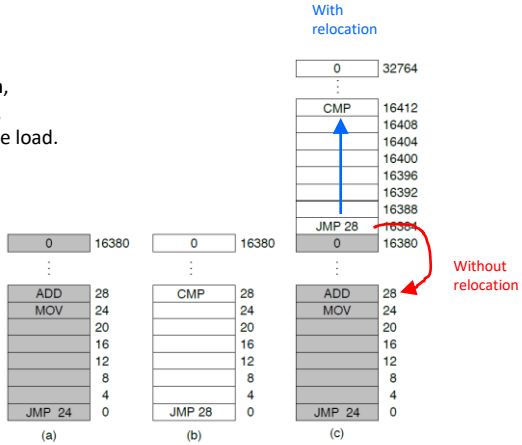


Figure 3-2. Illustration of the relocation problem. (a) A 16-KB program. (b) Another 16-KB program. (c) The two programs loaded consecutively into memory.

Memory Management (6)

However, even with no memory abstraction, it is possible to run multiple programes at the same time. What the operating system has to do is save the entire content of memory to a file on nonvolatile storage, then bring in and run the next program. As long as there is only one program at a time in memory, the are no conflits.
This operation is called (**swapping**).

There are not direct addressing of physical memory in mainframes, minicomputers, desktops, notebooks, smartphones, but it is quite common in embedded and smart cards systems.

Devices such as radios, washing machines, and microwave ovens are all full of software (in ROM) these days, and in most cases the software addresses absolute memory. These works because all programs are known in advance and users are not free to run their own software in their toaster.

A simpler smartphone, can be in this group, because their operating system is like a library linked with the application program and provides system calls for performing I/O and other common tasks.

A Memory Abstraction: **Address Space**

An address space is the set of addresses that a process can use to address memory.

Dynamic Relocation

Maps each process' address space onto different part of physical memory. Used on machines ranging from the CDC6600 to the Intel 8088 , equip each CPU with two hardware registers called **Base** and **Limit**. When this are used, programs are loaded into consecutive memory locations wherever there is room and without relocation during loading.

When a process is run, the base register is loaded with the physical address where its program begins in memory and the limit register is loaded with the length of the program.

Problems:

A disadvantage of relocation using base and limit registers is the **need to perform** an **addition and a comparison** on every memory reference. Comparisons-CMP are fast, but additions-ADD are slow.

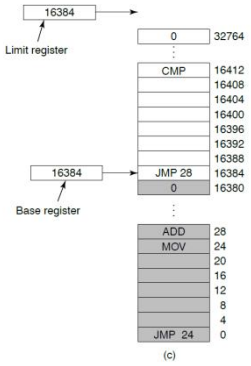


Figure 3-3. Base and limit registers can be used to give each process a separate address space.

Memory Management (8)

In pratice, the total amount of RAM needed by all processes is often much more than can fit in memory.

On a typical Windows, MacOS, or Linux system, something like 50-100 processes or more may be started up as soon as the computer is booted.

Examples:

- Background processes checking for incoming mail;
- Background processes checking for incoming networks connections;
- Some Windows applications have processes started at boot just for checking new updates;
- etc.

A serious user applications like Photoshop, can require almost 1Gbyte just to boot and many gigabyte once they start processing data.
Consequently, keeping all processes in memory all the time requires a huge amount of memory and can not be done if there is insufficient memory.

Two general approaches to dealing with memory overload:

Swapping – consists of bringing in each process in its entirety, running it for a while, then putting it back on nonvolatile storage (disk or SSD). They could wake up periodicaly to do their work, and go to sleep again if not needed.

Virtual Memory – allows programs to run even when they are only partially in main memory.
The basic idea of virtual memory is that each program has its own address space.

Memory Management - Swapping (1)

Example:

(a) Initially only Process *A* in memory;

(b) Process *B* are created;

(c) Process *C* are created or swapped in from nonvolatile storage;

(d) *A* is swapped out to nonvolatile storage;

(e) *D* comes in;

(f) *B* termines or is swapped out;

(g) *A* comes in again, since *A* is now in a different address location, addresses contained in it must be relocate. Relocation can be made by software when it is swapped in, or (more likely) by hardware during program execution.

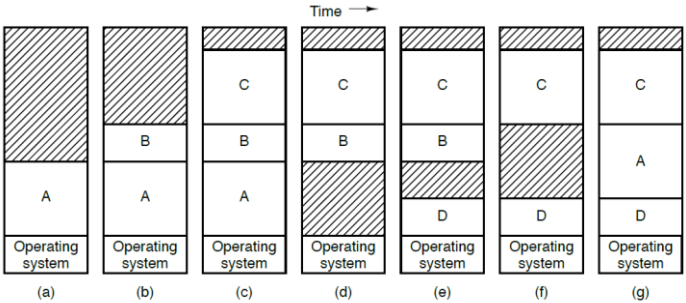


Figure 3-4. Memory allocation changes as processes come into memory and leave it. The shaded regions are unused memory.

Shaded regions are unused memory. Swapping creates multiple holes in memory.

It is possible to combine them all into one big area, moving all processes downward as far as possible – **memory compaction**.

Memory Management - Swapping (2)

Some **processes have a fixed size**, so the allocation in memory is simple: the operating system allocates exactly what is needed.

Some other **processes have data segments that can grow**.

If it is expected that most processes will grow as they run, it is probably a good idea to allocate a little extra memory whenever a process is swapped in or moved, to reduce the overhead associated with processes that no longer fit in their allocated memory.

If a process can have two growing segments, as an example:

- **Data segment** being used as a heap for variables that are dynamically allocated and released;

- **Stack segment** for normal local variables and return addresses.

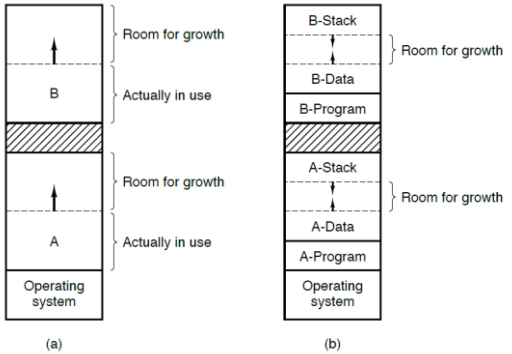


Figure 3-5. (a) Allocating space for a growing data segment. (b) Allocating space for a growing stack and a growing data segment.

Managing Free Memory

When **memory is assigned dynamically,** the **operating system must manage it**.

There are two ways to keep track of memory usage: **Bitmaps** and **free lists**.

The size of the allocation unit is an important design issue.
The smaller the allocation unit, the larger the bitmap.

Example:
Allocation unit = 4 bytes
32 bits memory = 1 bit of the map

A memory of 32*n* bits will use *n* map bits, so the bitmap will take only 1/32 of memory.

A bitmap provides a simple way to keep track of memory, but searching the bitmap for space to put a new run could be a slow operation.

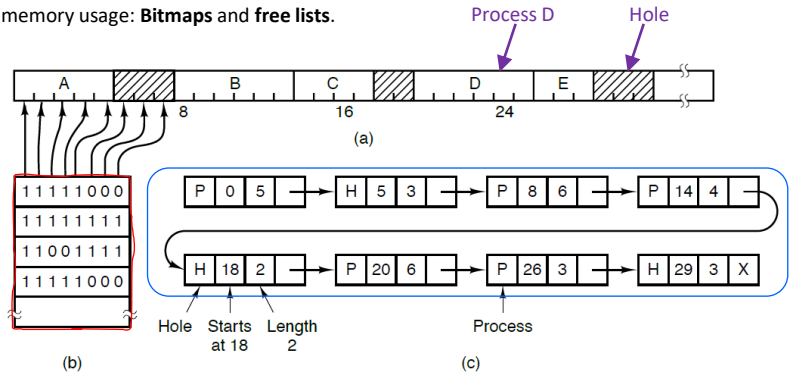


Figure 3-6. (a) A part of memory with five processes and three holes. The tick marks show the memory allocation units. The shaded regions (0 in the bitmap) are free. (b) The corresponding bitmap. (c) The same information as a list.

When the processes and holes are kept on a list sorted by address, several algorithms can be used to allocate memory for a created process (or an existing process being swapped in from disk or SSD).

Assuming that the memory manager knows how much memory to allocate. The simple algorithm is First Fit.

First Fit: memory manager scans along the list of segments until it finds a hole that is big enough. Its fast.

Next Fit: it works the same way as first fit, except that it keeps track of where it is whenever it finds a suitable hole.

Best Fit: searches the entires list, from beginning to end, and takes the smallest hole that is adquate.

Worst Fit: always takes the largest available hole.

Not a very good idea…

Quick Fit: maintains separate lists for some of the more common sizes requested. For example, it might have a table with *n* entries, in which the first entry is a pointer to the head of a list of 4-KB holes, the second entry is a pointer to a list of 8-KB holes,…, and so on. Finding a hole of the required size is extremely fast.

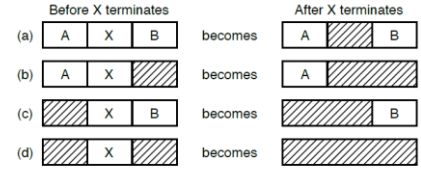


Figure 3-7. Four neighbor combinations for the terminating process, *X*.

Nevertheless,

if merging is not done, memory will quickly fragment into a large number of small holes into which no processes fit.

Memory Management (9)



QUESTÕES?

Daniel Castro
dcastro@ualg.pt

